

Availability violations — explained (the “17 violations”)

This note explains the message you may see on the **Integrity** page, in the “**Teacher availability — verification**” panel:

Unavailable slots: **17 violation(s)** detected

It also documents how the system now **classifies** those violations so you can tell at a glance which ones are expected and which (if any) need attention.

1. What the check does

On every run, the pipeline produces a factual proof (`config/availability_verification.json`) that the **produced** schedule respects the parameters set in Teacher Availability Configuration. Three things are verified:

Check	Type	Meaning
Unavailable slots	HARD	No scheduled group lands on a slot that is blocked for its subject (i.e. all professors of that subject are unavailable at that day/slot).
Preferred time range	SOFT	% of sessions placed inside each teacher’s preferred range.
Max lab days / week	SIGNAL	Days actually used vs the teacher’s cap.

The “17 violations” come from the **first** check (Unavailable slots).

2. Why violations can appear — and why most are expected

A slot is “blocked” for a subject when **every** professor teaching that subject declared themselves unavailable at that day/time. During group formation (`pipeline.py` → `form_groups`), the solver removes those blocked slots from the candidate list for the subject.

But there is a deliberate safety valve. If removing the blocked slots would leave **zero feasible slots** for a subject, the constraint is relaxed on purpose so that the subject can still be scheduled. Without this relaxation the subject would simply have **no schedule at all**, which is worse than a flagged, documented overlap.

This is fully consistent with the core project principle:

The system validates, it does not decide.

Hard conflicts that would make scheduling impossible are **signaled, not blocked**.

So the vast majority — and usually **all** — of the 17 violations are these **expected relaxations**.

3. The new classification (relaxed vs unexpected)

The verifier now tags **each** violation:

- **relaxed (expected)** — the subject is in `RELAXED_PROF_CONSTRAINT_SUBJECTS`, meaning enforcing the unavailability would have left no feasible slot. This is expected and acceptable.
- **unexpected** — a violation **not** explained by a documented relaxation. Only these warrant investigation.

The panel status reflects this:

Status	Condition	UI
ok	0 violations	green success
relaxed	all violations are documented relaxations	amber warning (“expected”)
violated	at least one <code>unexpected</code> violation	red error

The JSON now also carries `relaxed_count`, `unexpected_count` and `relaxed_subjects` so the figures are auditable.

4. Which subjects are typically affected

In the current dataset the flagged placements concentrate on afternoon slots (15:00–17:00 / 17:00–19:00) of subjects whose declared professors are all unavailable in those ranges, for example:

- S1 — Termodinámica
- S1 — Electrotecnia
- S1 — Mecánica

- S1 — Regulación Automática
- S2 — Resistencia de Materiales

(The exact list is printed in the panel table and in the JSON.)

These subjects map to afternoon course blocks (curso 2/4 → afternoon), so the only feasible lab windows collide with the declared unavailability — hence the relaxation.

5. How to remove a violation (if you want zero)

A `relaxed` violation is not a bug; it is the system telling you the inputs are over-constrained. To make it disappear you can do **one** of:

1. **Relax a teacher's unavailability** for the affected subject so at least one afternoon slot becomes free again.
2. **Add a room or a time slot** so an alternative feasible window exists.
3. **Accept it** — the overlap is documented and the schedule remains valid.

After changing any input, re-run the pipeline; the verification regenerates and the counts update automatically.

6. Where this lives in the code

- `pipeline.py` → `verify_availability_constraints()` — builds the proof and the `relaxed` / `unexpected` classification.
- `pipeline.py` → `form_groups()` — records relaxed subjects in `RELAXED_PROF_CONSTRAINT_SUBJECTS` when the prof-busy constraint would empty all slots.
- `app.py` (Integrity page) — renders the panel with the English explanation and the per-violation table.